Modelagem de Dados — Sistema Vida Plena (SVP)

1. Introdução e premissas

A modelagem organiza os dados do SVP em 42 tabelas, agrupadas em 8 módulos, com o animal como paciente e o tutor como titular dos dados pessoais.

Premissas adotadas

- O paciente é o animal; o responsável é o tutor (pessoa física). Um tutor pode ter vários animais, e cada animal tem um tutor responsável.
- A empresa é uma rede: agenda, estoque e faturamento são registrados por unidade.
- Todo atendimento nasce de um agendamento. Encaixes e chegadas sem hora marcada são lançados pela recepção como agendamento no ato.
- Laudos, exames e imagens ficam no repositório de arquivos; o banco guarda só o caminho e o hash de cada arquivo.

Ajustes sugeridos à etapa 1

- CID é a classificação de doenças humanas. O modelo usa um catálogo próprio de diagnósticos veterinários (tabela `diagnostico`), que pode seguir uma nomenclatura padronizada como a VeNom.
- Na LGPD, dado sensível é dado de pessoa natural; a saúde do animal não se enquadra no Art. 5º, II. Os dados do tutor são dados pessoais, e os dados clínicos estão protegidos pelo sigilo profissional do médico-veterinário. O modelo protege os dois com o mesmo rigor.
- A etapa 1 diz que os clientes são só particulares, mas lista operadoras de convênio como parceiras. O modelo trata convênio como forma de pagamento opcional.

2. Modelo conceitual

O cadastro diz quem é o animal e quem responde por ele, a agenda marca o atendimento e o prontuário registra o que aconteceu; os demais módulos cobram, abastecem e controlam o acesso.

Módulo	Entidades	O que representam
Cadastro	tutor, consentimento, animal, especie, raca, alergia, convenio, animal_convenio	Quem é o animal, quem responde por ele, o que o tutor autorizou e qual plano cobre o animal

Módulo	Entidades	O que representam
Agenda	unidade, especialidade, veterinario, veterinario_especialidade, agendamento	Onde, com quem e quando o animal será atendido, presencialmente ou por teleconsulta
Prontuário	triagem, atendimento, diagnostico, atendimento_diagnostico, evolucao, prescricao, item_prescricao	Sinais vitais, anamnese, diagnósticos, evolução e receitas de cada atendimento
Exames	laboratorio, tipo_exame, solicitacao_exame, documento	Pedido, coleta própria, envio ao laboratório terceirizado e laudo digitalizado
Programas crônicos	programa_cronico, inscricao_programa	Acompanhamento contínuo de animais com doença crônica
Financeiro	servico, fatura, item_fatura, pagamento, nota_fiscal	Tabela de preços, cobrança, recebimento e nota fiscal de serviço
Estoque	fornecedor, insumo, lote, movimentacao_estoque	Medicamentos e descartáveis por lote, validade e unidade
Equipe e acesso	funcionario, usuario, perfil, permissao, perfil_permissao, usuario_perfil, log_auditoria	Quem usa o sistema, o que cada perfil pode fazer e o registro de cada acesso

Regras de negócio

- **RN01** — Todo animal tem um tutor responsável; um tutor pode ter vários animais.
- **RN02** — CPF do tutor e microchip do animal são únicos no sistema.
- **RN03** — A raça, quando informada, precisa pertencer à espécie do animal; sem raça, o animal é SRD.
- **RN04** — O agendamento só aceita um veterinário que tenha a especialidade escolhida.
- **RN05** — Um veterinário não pode ter dois agendamentos ativos no mesmo horário.
- **RN06** — O status do agendamento vai de AGENDADO para CONFIRMADO e REALIZADO, ou termina em CANCELADO ou FALTOU.
- **RN07** — Cada agendamento gera no máximo uma triagem e um atendimento; a teleconsulta dispensa triagem.
- **RN08** — O prontuário não é apagado: correções viram nova evolução ou alteração registrada no log.
- **RN09** — Cada atendimento tem no máximo um diagnóstico principal.

- **RN10** — O pedido de exame passa por SOLICITADO, COLETADO, ENVIADO e CONCLUIDO; o laudo fica em `documento`.
- **RN11** — A fatura guarda o valor praticado no dia, mesmo que a tabela de preços mude depois.
- **RN12** — Pagamento por convênio exige a operadora. O sistema nunca guarda número de cartão, só o código de autorização.
- **RN13** — Medicamento usado no atendimento dá baixa no lote e fica ligado ao atendimento.
- **RN14** — Teleconsulta, comunicações e envio de amostras ao laboratório exigem consentimento ativo do tutor para aquela finalidade.
- **RN15** — Toda leitura, inclusão, alteração ou exclusão de dado clínico gera um registro no log de auditoria, que não pode ser alterado.

3. Diagrama entidade-relacionamento

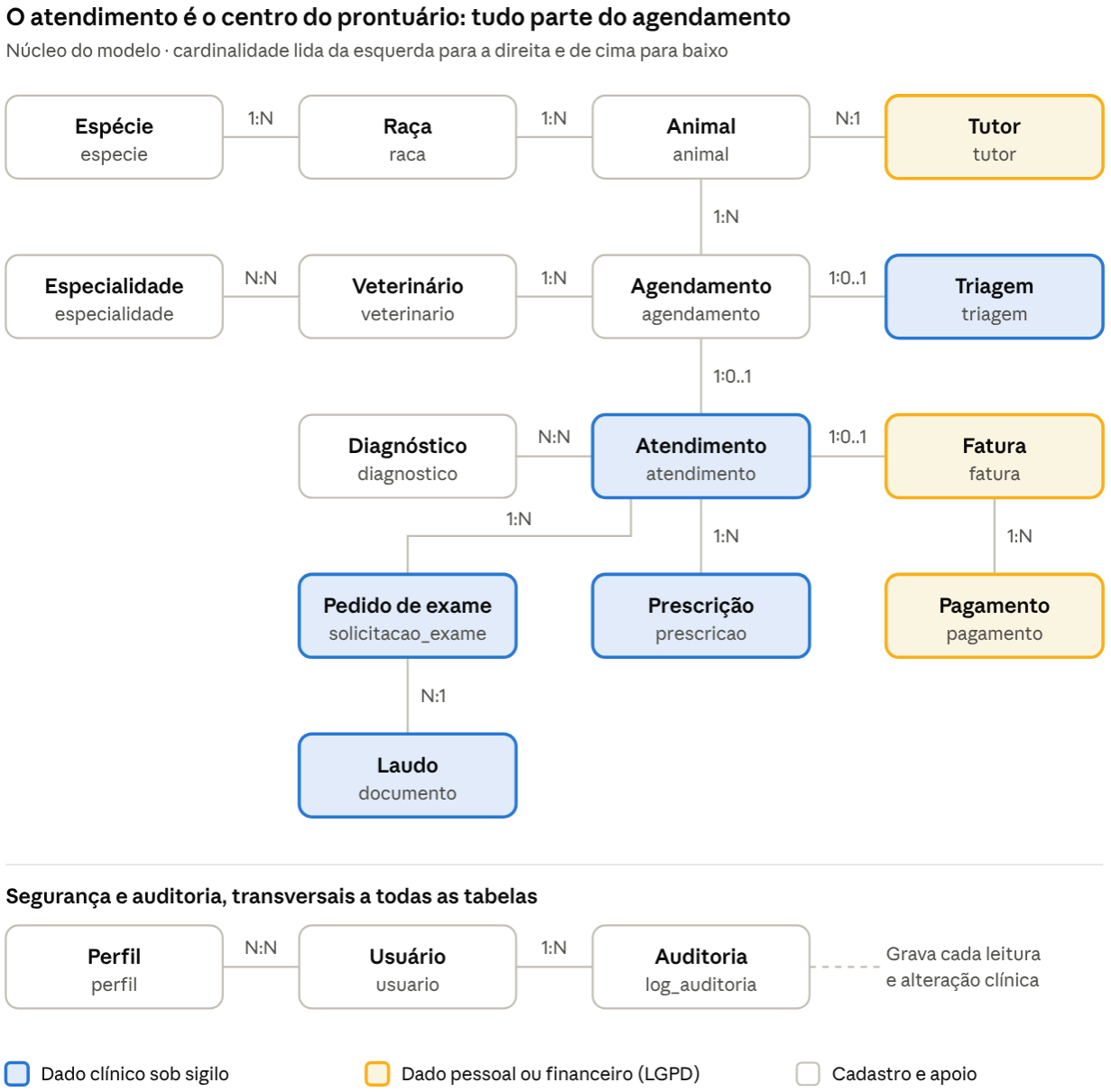


diagrama ER · 18 entidades centrais

O animal chega por um agendamento, que gera no máximo uma triagem e um atendimento; diagnóstico, prescrição, exames e fatura partem do atendimento. O diagrama mostra o núcleo — a fatura também referencia o tutor pagador, e estoque, programas crônicos, consentimento e nota fiscal estão no modelo lógico da seção 4.

4. Modelo lógico

Cada entidade vira uma tabela com chave primária substituta (`id_…`), e cada relacionamento N:N vira uma tabela associativa; o modelo está na 3ª forma normal. Na coluna FK, a seta indica a tabela referenciada.

Cadastro

Tabela	PK	FK	Demais atributos
tutor	id_tutor	—	nome, cpf (único), data_nascimento, telefone, email, logradouro, numero, complemento, bairro, cidade, uf, cep, contato_validado_em, criado_em, atualizado_em, anonimizado_em
consentimento	id_consentimento	id_tutor → tutor	finalidade, versao_termo, aceito_em, revogado_em, canal
especie	id_especie	—	nome (único)
raca	id_raca	id_especie → especie	nome
animal	id_animal	id_tutor → tutor; id_especie → especie; (id_raca, id_especie) → raca	nome, sexo, data_nascimento, nascimento_estimado, porte, microchip (único), castrado, data_obito, criado_em
alergia	id_alergia	id_animal → animal; registrada_por → usuario	substancia, gravidade, observacao, registrada_em
convenio	id_convenio	—	razao_social, cnpj (único), ativo
animal_convenio	(id_animal, id_convenio)	id_animal → animal; id_convenio → convenio	numero_carteira, validade

Agenda

Tabela	PK	FK	Demais atributos
unidade	id_unidade	—	nome, cnpj (único), telefone, logradouro, numero, bairro, cidade, uf, cep, ativa
especialidade	id_especialidade	—	nome (único)
veterinario	id_veterinario	id_veterinario → funcionario	crmv, uf_crmv
veterinario_especialidade	(id_veterinario, id_especialidade)	id_veterinario → veterinario; id_especialidade → especialidade	—
agendamento	id_agendamento	id_animal → animal; (id_veterinario, id_especialidade) → veterinario_especialidade; id_unidade → unidade; criado_por → usuario	data_hora, duracao_min, modalidade, status, motivo, criado_em, chegada_em

Prontuário

Tabela	PK	FK	Demais atributos
triagem	id_triagem	id_agendamento → agendamento (único); id_funcionario → funcionario	realizada_em, peso_kg, temperatura_c, freq_cardiaca, freq_respiratoria, queixa_principal, prioridade

Tabela	PK	FK	Demais atributos
atendimento	id_atendimento	id_agendamento → agendamento (único); id_veterinario → veterinario	inicio_em, fim_em, anamnese, exame_fisico, conduta, finalizado
diagnostico	id_diagnostico	—	codigo (único), descricao, cronico
atendimento_diagnostico	(id_atendimento, id_diagnostico)	id_atendimento → atendimento; id_diagnostico → diagnostico	tipo
evolucao	id_evolucao	id_animal → animal; id_atendimento → atendimento (opcional); id_funcionario → funcionario	registrada_em, texto
prescricao	id_prescricao	id_atendimento → atendimento	emitida_em, observacao
item_prescricao	id_item	id_prescricao → prescricao; id_insumo → insumo (opcional)	medicamento_texto, dose, via, frequencia, duracao_dias

Exames

Tabela	PK	FK	Demais atributos
laboratorio	id_laboratorio	—	razao_social, cnpj (único), email_integracao, ativo
tipo_exame	id_tipo_exame	id_servico → servico	nome (único), material, prazo_dias
documento	id_documento	id_animal → animal; enviado_por → usuario	tipo, nome_arquivo, caminho, hash_sha256, enviado_em

Tabela	PK	FK	Demais atributos
solicitacao_exame	id_solicitacao	id_atendimento → atendimento; id_tipo_exame → tipo_exame; id_laboratorio → laboratorio; coletado_por → funcionario; id_documento → documento	status, coletado_em, resultado_em, resultado_resumo

Programas crônicos

Tabela	PK	FK	Demais atributos
programa_cronico	id_programa	id_servico → servico	nome (único), descricao, intervalo_retorno_dias
inscricao_programa	id_inscricao	id_animal → animal; id_programa → programa_cronico; id_veterinario → veterinario	inicio, fim, status

Financeiro

Tabela	PK	FK	Demais atributos
servico	id_servico	—	descricao (único), tipo, valor_base, ativo
fatura	id_fatura	id_tutor → tutor; id_atendimento → atendimento (opcional, único); id_unidade → unidade	emitida_em, vencimento, desconto, status
item_fatura	id_item_fatura	id_fatura → fatura; id_servico → servico	quantidade, valor_unitario

Tabela	PK	FK	Demais atributos
pagamento	id_pagamento	id_fatura → fatura; id_convenio → convenio (opcional)	forma, valor, parcelas, codigo_autorizacao, pago_em
nota_fiscal	id_nota	id_fatura → fatura (único)	numero, serie, codigo_verificacao, emitida_em, valor

Estoque

Tabela	PK	FK	Demais atributos
fornecedor	id_fornecedor	—	razao_social, cnpj (único), telefone, email
insumo	id_insumo	—	nome (único), categoria, unidade_medida, controlado, estoque_minimo
lote	id_lote	id_insumo → insumo; id_fornecedor → fornecedor; id_unidade → unidade	numero_lote, validade, custo_unitario
movimentacao_estoque	id_movimentacao	id_lote → lote; id_atendimento → atendimento (opcional); id_usuario → usuario	tipo, quantidade, realizada_em, motivo

Equipe e acesso

Tabela	PK	FK	Demais atributos
funcionario	id_funcionario	id_unidade → unidade	nome, cpf (único), cargo, email (único), data_admissao, data_desligamento
usuario	id_usuario	id_funcionario → funcionario (único)	login (único), senha_hash, mfa_ativo, ativo, ultimo_acesso, criado_em
perfil	id_perfil	—	nome (único), descricao
permissao	id_permissao	—	recurso, acao
perfil_permissao	(id_perfil, id_permissao)	id_perfil → perfil; id_permissao → permissao	—
usuario_perfil	(id_usuario, id_perfil)	id_usuario → usuario; id_perfil → perfil	—
log_auditoria	id_log	id_usuario → usuario	ocorrido_em, acao, tabela, id_registro, dados_antes, dados_depois, ip_origem, justificativa

Relacionamentos e cardinalidades

Relacionamento	Cardinalidade	Como é implementado
tutor – animal	1:N	animal.id_tutor
tutor – consentimento	1:N	consentimento.id_tutor
especie – raca	1:N	raca.id_especie
animal – alergia	1:N	alergia.id_animal
animal – convenio	N:N	tabela animal_convenio
funcionario – veterinario	1:0..1	especialização: veterinario.id_veterinario é o próprio id_funcionario
veterinario – especialidade	N:N	tabela veterinario_especialidade

Relacionamento	Cardinalidade	Como é implementado
animal – agendamento	1:N	agendamento.id_animal
agendamento – triagem	1:0..1	triagem.id_agendamento único
agendamento – atendimento	1:0..1	atendimento.id_agendamento único
atendimento – diagnostico	N:N	tabela atendimento_diagnostico
atendimento – prescricao – item_prescricao	1:N e 1:N	prescricao.id_atendimento; item_prescricao.id_prescricao
atendimento – solicitacao_exame	1:N	solicitacao_exame.id_atendimento
documento – solicitacao_exame	1:N	solicitacao_exame.id_documento (um laudo pode cobrir vários exames)
animal – programa_cronico	N:N	tabela inscricao_programa
atendimento – fatura	1:0..1	fatura.id_atendimento único
fatura – item_fatura / pagamento	1:N	item_fatura.id_fatura; pagamento.id_fatura
fatura – nota_fiscal	1:0..1	nota_fiscal.id_fatura único
insumo – lote – movimentacao_estoque	1:N e 1:N	lote.id_insumo; movimentacao_estoque.id_lote
usuario – perfil	N:N	tabela usuario_perfil
usuario – log_auditoria	1:N	log_auditoria.id_usuario

Normalização

- **1FN** — todos os atributos são atômicos; o endereço é dividido em campos e não há listas dentro de uma coluna.
- **2FN** — nas tabelas de chave composta, cada atributo depende da chave inteira (ex.: `tipo` em atendimento_diagnostico depende do par atendimento + diagnóstico).
- **3FN** — nada derivado é gravado: o total da fatura e o saldo do estoque são calculados nas visões `vw_fatura_total` e `vw_saldo_lote`.
- **Redundância consciente** — item_fatura.valor_unitario repete o preço do serviço para preservar o valor cobrado no dia (RN11).

5. Dicionário de dados

O dicionário detalha as seis tabelas centrais; as demais seguem o mesmo padrão e estão completas no script da seção 7.

tutor

Campo	Tipo	Obrigatório	Restrição	Descrição
id_tutor	INT (identidade)	Sim	PK	Identificador interno
nome	VARCHAR(120)	Sim	—	Nome completo do tutor
cpf	CHAR(11)	Sim	Único; só dígitos	Chave natural que impede cadastro duplicado
data_nascimento	DATE	Não	—	Data de nascimento do tutor
telefone	VARCHAR(15)	Sim	—	Contato principal, com DDD
email	VARCHAR(120)	Não	—	Contato e envio de laudos
logradouro, numero, complemento, bairro, cidade	VARCHAR	Não	—	Endereço
uf / cep	CHAR(2) / CHAR(8)	Não	—	Estado e CEP
contato_validado_em	DATE	Não	—	Última vez que a recepção confirmou telefone e endereço
criado_em / atualizado_em	TIMESTAMPTZ	Sim	Padrão now()	Controle do cadastro

Campo	Tipo	Obrigatório	Restrição	Descrição
anonimizado_em	TIMESTAMPTZ	Não	—	Quando os dados pessoais foram anonimizados pela política de descarte

animal

Campo	Tipo	Obrigatório	Restrição	Descrição
id_animal	INT (identidade)	Sim	PK	Identificador do paciente
id_tutor	INT	Sim	FK → tutor	Responsável pelo animal
id_especie	INT	Sim	FK → especie	Cão, gato, equino, bovino etc.
id_raca	INT	Não	FK (id_raca, id_especie) → raca	Raça da mesma espécie; vazio = SRD
nome	VARCHAR(60)	Sim	Único por tutor e espécie, entre animais vivos	Nome do animal
sexo	CHAR(1)	Sim	M ou F	Sexo
data_nascimento	DATE	Não	—	Nascimento, real ou estimado
nascimento_estimado	BOOLEAN	Sim	Padrão falso	Indica se a data é estimada
porte	CHAR(1)	Sim	P, M ou G	Pequeno, médio ou grande porte
microchip	VARCHAR(15)	Não	Único	Identificação eletrônica

Campo	Tipo	Obrigatório	Restrição	Descrição
castrado	BOOLEAN	Não	—	Situação reprodutiva
data_obito	DATE	Não	—	Preenchida no óbito; o histórico é mantido

agendamento

Campo	Tipo	Obrigatório	Restrição	Descrição
id_agendamento	INT (identidade)	Sim	PK	Identificador
id_animal	INT	Sim	FK → animal	Paciente
id_veterinario, id_especialidade	INT	Sim	FK composta → veterinario_especialidade	Veterinário e especialidade que ele atende
id_unidade	INT	Sim	FK → unidade	Clínica da rede
data_hora	TIMESTAMPTZ	Sim	Único por veterinário, exceto cancelados	Início marcado
duracao_min	SMALLINT	Sim	> 0; padrão 30	Duração prevista
modalidade	VARCHAR(12)	Sim	PRESENCIAL ou TELECONSULTA	Tipo de consulta
status	VARCHAR(12)	Sim	AGENDADO, CONFIRMADO, CANCELADO, FALTOU, REALIZADO	Situação
motivo	VARCHAR(200)	Não	—	Motivo informado na marcação
criado_por	INT	Sim	FK → usuario	Quem marcou
chegada_em	TIMESTAMPTZ	Não	—	Chegada registrada na recepção; base do tempo de espera

atendimento

Campo	Tipo	Obrigatório	Restrição	Descrição
id_atendimento	INT (identidade)	Sim	PK	Registro do prontuário
id_agendamento	INT	Sim	FK → agendamento; único	Agendamento que originou o atendimento
id_veterinario	INT	Sim	FK → veterinario	Quem de fato atendeu (pode diferir do agendado)
inicio_em	TIMESTAMPTZ	Sim	—	Início do atendimento
fim_em	TIMESTAMPTZ	Não	⩾ inicio_em	Fim do atendimento
anamnese	TEXT	Sim	—	Histórico relatado pelo tutor
exame_fisico	TEXT	Não	—	Achados do exame clínico
conduta	TEXT	Não	—	Plano terapêutico e orientações
finalizado	BOOLEAN	Sim	Se verdadeiro, exige fim_em	Indica prontuário fechado

pagamento

Campo	Tipo	Obrigatório	Restrição	Descrição
id_pagamento	INT (identidade)	Sim	PK	Identificador
id_fatura	INT	Sim	FK → fatura	Fatura quitada (total ou parcial)

Campo	Tipo	Obrigatório	Restrição	Descrição
forma	VARCHAR(15)	Sim	PIX, CREDITO, DEBITO, DINHEIRO, BOLETO, CONVENIO	Forma de pagamento
valor	NUMERIC(10,2)	Sim	> 0	Valor recebido
parcelas	SMALLINT	Sim	1 a 12; padrão 1	Parcelamento no cartão
codigo_autorizacao	VARCHAR(40)	Não	—	NSU ou ID da transação; nunca o número do cartão
id_convenio	INT	Condicional	FK → convenio; obrigatório só se forma = CONVENIO	Operadora pagadora
pago_em	TIMESTAMPTZ	Sim	Padrão now()	Data e hora do recebimento

log_auditoria

Campo	Tipo	Obrigatório	Restrição	Descrição
id_log	BIGINT (identidade)	Sim	PK	Identificador sequencial
id_usuario	INT	Não	FK → usuario	Quem agiu; vazio em login com usuário inexistente
ocorrido_em	TIMESTAMPTZ	Sim	Padrão now()	Momento do evento

Campo	Tipo	Obrigatório	Restrição	Descrição
acao	VARCHAR(12)	Sim	LOGIN, LOGIN_FALHA, LOGOUT, LEITURA, INSERCAO, ALTERACAO, EXCLUSAO, EXPORTACAO	Tipo de evento
tabela / id_registro	VARCHAR(60) / BIGINT	Não	—	Registro afetado
dados_antes / dados_depois	JSONB	Não	—	Conteúdo antes e depois da alteração
ip_origem	INET	Não	—	Endereço de origem
justificativa	VARCHAR(200)	Não	Obrigatória em acesso de emergência	Motivo do acesso excepcional

6. Classificação dos dados (LGPD) e controle de acesso

Dados pessoais do tutor e dados clínicos do animal recebem a mesma proteção: acesso só por perfil autorizado e registro de cada leitura e alteração.

Classificação

Categoria	Onde está no modelo	Por que proteger	Proteção no modelo
Dados pessoais do tutor	tutor, consentimento	Dado pessoal (LGPD, Art. 5º, I)	Acesso por perfil; consentimento por finalidade; anonimização (anonimizado_em)
Dados clínicos do animal	triagem, atendimento, atendimento_diagnostico, evolucao, prescricao, item_prescricao, solicitacao_exame, documento, alergia	Sigilo profissional do médico-veterinário; ligados a um tutor identificado	Só perfis clínicos; log de leitura e alteração; exclusão bloqueada

Categoria	Onde está no modelo	Por que proteger	Proteção no modelo
Dados financeiros	fatura, item_fatura, pagamento, nota_fiscal, animal_convenio	Dado pessoal; risco de fraude	Sem número de cartão; só código de autorização; acesso de financeiro e recepção
Dados da equipe	funcionario	Dado pessoal do colaborador	Diretoria administrativa e TI
Credenciais	usuario.senha_hash	Porta de entrada do sistema	Hash bcrypt ou Argon2; MFA; usuário desativado no desligamento
Trilha de auditoria	log_auditoria	Prova de conformidade	Somente inclusão; UPDATE e DELETE revogados

Perfis de acesso (RBAC)

E = lê e edita · L = só lê · R = só relatórios agregados, sem identificar o tutor · — = sem acesso.

Perfil	Cadastro	Agenda	Prontuário	Exames	Financeiro	Estoque	Log
Veterinário	L	L	E	E	—	L	—
Enfermagem	L	L	E (triagem e evolução)	E (coleta)	—	E (baixa)	—
Recepção	E	E	—	L (status)	E (recebimento)	—	—
Financeiro	L	L	—	—	E	L	—
Diretoria	R	R	R	R	R	R	—
Auditoria interna	L	L	L (com justificativa)	L	L	L	L
Administrador de TI	—	—	—	—	—	—	L

O administrador de TI gerencia usuários e perfis, mas não lê dados de negócio; um acesso técnico de emergência exige justificativa e fica no log. As operadoras de convênio não têm login: recebem só os dados de faturamento exportados, e cada exportação é registrada.

Retenção e descarte

Prontuários e documentos ficam guardados pelo prazo da política de retenção da clínica, alinhado às normas do CFMV. Encerrado o prazo, ou a pedido do tutor quando não houver obrigação de guarda, os dados pessoais do tutor são anonimizados e o histórico clínico permanece sem identificação.

7. Modelo físico (PostgreSQL)

O script cria as 42 tabelas na ordem das dependências, com as restrições de integridade, a auditoria automática e as visões dos indicadores; ele foi executado e testado em PostgreSQL. As regras RN03, RN04, RN05, RN09 e RN12 são garantidas pelo próprio banco, e não só pela aplicação.

```
-- ========================================================================
-- Sistema Vida Plena (SVP) — Modelo físico
-- PostgreSQL 12 ou superior
-- ========================================================================

-- ------------------------------------------------------------------------
-- 1. Equipe e acesso
-- ------------------------------------------------------------------------
CREATE TABLE unidade (
  id_unidade   INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nome         VARCHAR(100) NOT NULL,
  cnpj         CHAR(14)     NOT NULL UNIQUE,
  telefone     VARCHAR(15),
  logradouro   VARCHAR(150),
  numero       VARCHAR(10),
  bairro       VARCHAR(80),
  cidade       VARCHAR(80)  NOT NULL,
  uf           CHAR(2)      NOT NULL,
  cep          CHAR(8),
  ativa        BOOLEAN      NOT NULL DEFAULT TRUE
);

CREATE TABLE funcionario (
  id_funcionario    INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_unidade        INT          NOT NULL REFERENCES unidade (id_unidade),
  nome              VARCHAR(120) NOT NULL,
  cpf               CHAR(11)     NOT NULL UNIQUE,
  cargo             VARCHAR(30)  NOT NULL
                    CHECK (cargo IN
('VETERINARIO','ENFERMEIRO','RECEPCIONISTA',
                                    'FINANCEIRO','TI','DIRETOR','AUDITOR')),
```

```
  email             VARCHAR(120) NOT NULL UNIQUE,
  data_admissao     DATE         NOT NULL,
  data_desligamento DATE,
  CHECK (data_desligamento IS NULL OR data_desligamento >= data_admissao)
);

CREATE TABLE veterinario (
  id_veterinario INT PRIMARY KEY REFERENCES funcionario (id_funcionario),
  crmv           VARCHAR(10) NOT NULL,
  uf_crmv        CHAR(2)     NOT NULL,
  UNIQUE (crmv, uf_crmv)
);

CREATE TABLE especialidade (
  id_especialidade INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nome             VARCHAR(80) NOT NULL UNIQUE
);

CREATE TABLE veterinario_especialidade (
  id_veterinario   INT REFERENCES veterinario (id_veterinario),
  id_especialidade INT REFERENCES especialidade (id_especialidade),
  PRIMARY KEY (id_veterinario, id_especialidade)
);

CREATE TABLE perfil (
  id_perfil INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nome      VARCHAR(40)  NOT NULL UNIQUE,
  descricao VARCHAR(200)
);

CREATE TABLE permissao (
  id_permissao INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  recurso      VARCHAR(60) NOT NULL,          -- ex.: 'prontuario', 'fatura'
  acao         VARCHAR(10) NOT NULL
               CHECK (acao IN ('LER','CRIAR','EDITAR','EXCLUIR','EXPORTAR')),
  UNIQUE (recurso, acao)
);

CREATE TABLE perfil_permissao (
  id_perfil    INT REFERENCES perfil (id_perfil),
  id_permissao INT REFERENCES permissao (id_permissao),
  PRIMARY KEY (id_perfil, id_permissao)
);

CREATE TABLE usuario (
  id_usuario     INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
  id_funcionario INT          NOT NULL UNIQUE REFERENCES funcionario
(id_funcionario),
  login          VARCHAR(50)  NOT NULL UNIQUE,
  senha_hash     VARCHAR(255) NOT NULL,       -- bcrypt/Argon2; nunca a senha
  mfa_ativo      BOOLEAN      NOT NULL DEFAULT FALSE,
  ativo          BOOLEAN      NOT NULL DEFAULT TRUE,
  ultimo_acesso  TIMESTAMPTZ,
  criado_em      TIMESTAMPTZ  NOT NULL DEFAULT now()
);

CREATE TABLE usuario_perfil (
  id_usuario INT REFERENCES usuario (id_usuario),
  id_perfil  INT REFERENCES perfil (id_perfil),
  PRIMARY KEY (id_usuario, id_perfil)
);

CREATE TABLE log_auditoria (
  id_log        BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_usuario    INT REFERENCES usuario (id_usuario),
  ocorrido_em   TIMESTAMPTZ NOT NULL DEFAULT now(),
  acao          VARCHAR(12) NOT NULL
                CHECK (acao IN ('LOGIN','LOGIN_FALHA','LOGOUT','LEITURA',

'INSERCAO','ALTERACAO','EXCLUSAO','EXPORTACAO')),
  tabela        VARCHAR(60),
  id_registro   BIGINT,
  dados_antes   JSONB,
  dados_depois  JSONB,
  ip_origem     INET,
  justificativa VARCHAR(200)                -- obrigatória em acesso de
emergência
);
CREATE INDEX ix_log_usuario_data ON log_auditoria (id_usuario, ocorrido_em);
CREATE INDEX ix_log_registro     ON log_auditoria (tabela, id_registro);

-- ---------------------------------------------------------------------
-- 2. Cadastro
-- ---------------------------------------------------------------------
CREATE TABLE tutor (
  id_tutor            INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nome                VARCHAR(120) NOT NULL,
  cpf                 CHAR(11)     NOT NULL UNIQUE CHECK (cpf ~ '^[0-9]
{11}$'),
  data_nascimento     DATE,
  telefone            VARCHAR(15)  NOT NULL,
  email               VARCHAR(120),
```

```
  logradouro          VARCHAR(150),
  numero              VARCHAR(10),
  complemento         VARCHAR(60),
  bairro              VARCHAR(80),
  cidade              VARCHAR(80),
  uf                  CHAR(2),
  cep                 CHAR(8),
  contato_validado_em DATE,
  criado_em           TIMESTAMPTZ  NOT NULL DEFAULT now(),
  atualizado_em       TIMESTAMPTZ  NOT NULL DEFAULT now(),
  anonimizado_em      TIMESTAMPTZ
);

CREATE TABLE consentimento (
  id_consentimento INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_tutor         INT         NOT NULL REFERENCES tutor (id_tutor),
  finalidade       VARCHAR(30) NOT NULL
                   CHECK (finalidade IN
('ATENDIMENTO','TELECONSULTA','COMUNICACAO',

'MARKETING','COMPARTILHAMENTO_LAB')),
  versao_termo     VARCHAR(10) NOT NULL,
  aceito_em        TIMESTAMPTZ NOT NULL,
  revogado_em      TIMESTAMPTZ,
  canal            VARCHAR(20) NOT NULL
                   CHECK (canal IN ('PRESENCIAL','APP','SITE','TELEFONE')),
  CHECK (revogado_em IS NULL OR revogado_em >= aceito_em)
);

CREATE TABLE especie (
  id_especie INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nome       VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE raca (
  id_raca    INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_especie INT         NOT NULL REFERENCES especie (id_especie),
  nome       VARCHAR(60) NOT NULL,
  UNIQUE (id_especie, nome),
  UNIQUE (id_raca, id_especie)               -- permite a FK composta em
animal
);

CREATE TABLE animal (
  id_animal           INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_tutor            INT         NOT NULL REFERENCES tutor (id_tutor),
```

```
  id_especie          INT         NOT NULL REFERENCES especie (id_especie),
  id_raca             INT,                    -- nulo = SRD
  nome                VARCHAR(60) NOT NULL,
  sexo                CHAR(1)     NOT NULL CHECK (sexo IN ('M','F')),
  data_nascimento     DATE,
  nascimento_estimado BOOLEAN     NOT NULL DEFAULT FALSE,
  porte               CHAR(1)     NOT NULL CHECK (porte IN ('P','M','G')),
  microchip           VARCHAR(15) UNIQUE,
  castrado            BOOLEAN,
  data_obito          DATE,
  criado_em           TIMESTAMPTZ NOT NULL DEFAULT now(),
  -- a raça precisa ser da mesma espécie do animal (RN03)
  FOREIGN KEY (id_raca, id_especie) REFERENCES raca (id_raca, id_especie)
);
CREATE INDEX ix_animal_tutor ON animal (id_tutor);
-- evita o mesmo animal cadastrado duas vezes para o mesmo tutor
CREATE UNIQUE INDEX ux_animal_tutor_nome
  ON animal (id_tutor, id_especie, lower(nome)) WHERE data_obito IS NULL;

CREATE TABLE alergia (
  id_alergia     INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_animal      INT          NOT NULL REFERENCES animal (id_animal),
  substancia     VARCHAR(100) NOT NULL,
  gravidade      VARCHAR(10)  NOT NULL CHECK (gravidade IN
('LEVE','MODERADA','GRAVE')),
  observacao     TEXT,
  registrada_por INT          NOT NULL REFERENCES usuario (id_usuario),
  registrada_em  TIMESTAMPTZ  NOT NULL DEFAULT now(),
  UNIQUE (id_animal, substancia)
);

CREATE TABLE convenio (
  id_convenio  INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  razao_social VARCHAR(120) NOT NULL,
  cnpj         CHAR(14)     NOT NULL UNIQUE,
  ativo        BOOLEAN      NOT NULL DEFAULT TRUE
);

CREATE TABLE animal_convenio (
  id_animal       INT REFERENCES animal (id_animal),
  id_convenio     INT REFERENCES convenio (id_convenio),
  numero_carteira VARCHAR(30) NOT NULL,
  validade        DATE,
  PRIMARY KEY (id_animal, id_convenio)
);
```

```
-- ---------------------------------------------------------------------
-- 3. Catálogo de serviços (base do faturamento)
-- ---------------------------------------------------------------------
CREATE TABLE servico (
  id_servico INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  descricao  VARCHAR(120)  NOT NULL UNIQUE,
  tipo       VARCHAR(15)   NOT NULL
             CHECK (tipo IN
('CONSULTA','TELECONSULTA','EXAME','PROCEDIMENTO',
                           'PROGRAMA','MATERIAL')),
  valor_base NUMERIC(10,2) NOT NULL CHECK (valor_base >= 0),
  ativo      BOOLEAN       NOT NULL DEFAULT TRUE
);

-- ---------------------------------------------------------------------
-- 4. Agenda
-- ---------------------------------------------------------------------
CREATE TABLE agendamento (
  id_agendamento   INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_animal        INT          NOT NULL REFERENCES animal (id_animal),
  id_veterinario   INT          NOT NULL,
  id_especialidade INT          NOT NULL,
  id_unidade       INT          NOT NULL REFERENCES unidade (id_unidade),
  data_hora        TIMESTAMPTZ  NOT NULL,
  duracao_min      SMALLINT     NOT NULL DEFAULT 30 CHECK (duracao_min > 0),
  modalidade       VARCHAR(12)  NOT NULL CHECK (modalidade IN
('PRESENCIAL','TELECONSULTA')),
  status           VARCHAR(12)  NOT NULL DEFAULT 'AGENDADO'
                   CHECK (status IN
('AGENDADO','CONFIRMADO','CANCELADO','FALTOU','REALIZADO')),
  motivo           VARCHAR(200),
  criado_por       INT          NOT NULL REFERENCES usuario (id_usuario),
  criado_em        TIMESTAMPTZ  NOT NULL DEFAULT now(),
  chegada_em       TIMESTAMPTZ,              -- chegada registrada na
recepção
  -- o veterinário precisa atender a especialidade escolhida (RN04)
  FOREIGN KEY (id_veterinario, id_especialidade)
    REFERENCES veterinario_especialidade (id_veterinario, id_especialidade)
);
CREATE INDEX ix_agendamento_animal ON agendamento (id_animal);
CREATE INDEX ix_agendamento_agenda ON agendamento (id_unidade, data_hora);
-- um veterinário não tem dois horários ativos iguais (RN05)
CREATE UNIQUE INDEX ux_agenda_vet_horario
  ON agendamento (id_veterinario, data_hora) WHERE status <> 'CANCELADO';

-- ---------------------------------------------------------------------
```

```
-- 5. Prontuário
-- ---------------------------------------------------------------------
CREATE TABLE triagem (
  id_triagem        INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_agendamento    INT          NOT NULL UNIQUE REFERENCES agendamento
(id_agendamento),
  id_funcionario    INT          NOT NULL REFERENCES funcionario
(id_funcionario),
  realizada_em      TIMESTAMPTZ  NOT NULL DEFAULT now(),
  peso_kg           NUMERIC(7,2) CHECK (peso_kg > 0),
  temperatura_c     NUMERIC(4,1),
  freq_cardiaca     SMALLINT,
  freq_respiratoria SMALLINT,
  queixa_principal  VARCHAR(300) NOT NULL,
  prioridade        VARCHAR(15)  NOT NULL
                    CHECK (prioridade IN
('EMERGENCIA','URGENTE','POUCO_URGENTE','NAO_URGENTE'))
);

CREATE TABLE atendimento (
  id_atendimento INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_agendamento INT         NOT NULL UNIQUE REFERENCES agendamento
(id_agendamento),
  id_veterinario INT         NOT NULL REFERENCES veterinario
(id_veterinario),
  inicio_em      TIMESTAMPTZ NOT NULL,
  fim_em         TIMESTAMPTZ,
  anamnese       TEXT        NOT NULL,
  exame_fisico   TEXT,
  conduta        TEXT,
  finalizado     BOOLEAN     NOT NULL DEFAULT FALSE,
  CHECK (fim_em IS NULL OR fim_em >= inicio_em),
  CHECK (NOT finalizado OR fim_em IS NOT NULL)
);

CREATE TABLE diagnostico (
  id_diagnostico INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  codigo         VARCHAR(15)  NOT NULL UNIQUE,  -- catálogo veterinário
padronizado
  descricao      VARCHAR(200) NOT NULL,
  cronico        BOOLEAN      NOT NULL DEFAULT FALSE
);

CREATE TABLE atendimento_diagnostico (
  id_atendimento INT REFERENCES atendimento (id_atendimento),
  id_diagnostico INT REFERENCES diagnostico (id_diagnostico),
```

```
  tipo           VARCHAR(10) NOT NULL CHECK (tipo IN
('PRINCIPAL','SECUNDARIO','SUSPEITA')),
  PRIMARY KEY (id_atendimento, id_diagnostico)
);
-- no máximo um diagnóstico principal por atendimento (RN09)
CREATE UNIQUE INDEX ux_diag_principal
  ON atendimento_diagnostico (id_atendimento) WHERE tipo = 'PRINCIPAL';

CREATE TABLE evolucao (
  id_evolucao    INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_animal      INT         NOT NULL REFERENCES animal (id_animal),
  id_atendimento INT         REFERENCES atendimento (id_atendimento),
  id_funcionario INT         NOT NULL REFERENCES funcionario
(id_funcionario),
  registrada_em  TIMESTAMPTZ NOT NULL DEFAULT now(),
  texto          TEXT        NOT NULL
);
CREATE INDEX ix_evolucao_animal ON evolucao (id_animal, registrada_em);

-- ------------------------------------------------------------------
-- 6. Estoque
-- ------------------------------------------------------------------
CREATE TABLE fornecedor (
  id_fornecedor INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  razao_social  VARCHAR(120) NOT NULL,
  cnpj          CHAR(14)     NOT NULL UNIQUE,
  telefone      VARCHAR(15),
  email         VARCHAR(120)
);

CREATE TABLE insumo (
  id_insumo      INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nome           VARCHAR(120)  NOT NULL UNIQUE,
  categoria      VARCHAR(15)   NOT NULL
                 CHECK (categoria IN
('MEDICAMENTO','DESCARTAVEL','VACINA','OUTRO')),
  unidade_medida VARCHAR(10)   NOT NULL,      -- ml, comprimido, unidade
  controlado     BOOLEAN       NOT NULL DEFAULT FALSE,
  estoque_minimo NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (estoque_minimo >= 0)
);

CREATE TABLE lote (
  id_lote        INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_insumo      INT           NOT NULL REFERENCES insumo (id_insumo),
  id_fornecedor  INT           NOT NULL REFERENCES fornecedor
(id_fornecedor),
```

```
  id_unidade     INT           NOT NULL REFERENCES unidade (id_unidade),
  numero_lote    VARCHAR(30)   NOT NULL,
  validade       DATE          NOT NULL,
  custo_unitario NUMERIC(10,2) NOT NULL CHECK (custo_unitario >= 0),
  UNIQUE (id_insumo, numero_lote, id_unidade)
);

CREATE TABLE movimentacao_estoque (
  id_movimentacao INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_lote         INT           NOT NULL REFERENCES lote (id_lote),
  tipo            VARCHAR(15)   NOT NULL
                  CHECK (tipo IN
('ENTRADA','SAIDA','PERDA','AJUSTE_ENTRADA','AJUSTE_SAIDA')),
  quantidade      NUMERIC(10,2) NOT NULL CHECK (quantidade > 0),
  id_atendimento  INT           REFERENCES atendimento (id_atendimento),  --
RN13
  id_usuario      INT           NOT NULL REFERENCES usuario (id_usuario),
  realizada_em    TIMESTAMPTZ   NOT NULL DEFAULT now(),
  motivo          VARCHAR(200)
);
CREATE INDEX ix_mov_lote ON movimentacao_estoque (id_lote);

-- ------------------------------------------------------------------
-- 7. Prescrição
-- ------------------------------------------------------------------
CREATE TABLE prescricao (
  id_prescricao  INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_atendimento INT         NOT NULL REFERENCES atendimento
(id_atendimento),
  emitida_em     TIMESTAMPTZ NOT NULL DEFAULT now(),
  observacao     TEXT
);

CREATE TABLE item_prescricao (
  id_item           INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_prescricao     INT          NOT NULL REFERENCES prescricao
(id_prescricao),
  id_insumo         INT          REFERENCES insumo (id_insumo),  -- nulo =
comprado fora
  medicamento_texto VARCHAR(150),
  dose              VARCHAR(40)  NOT NULL,
  via               VARCHAR(20)  NOT NULL,
  frequencia        VARCHAR(40)  NOT NULL,
  duracao_dias      SMALLINT     CHECK (duracao_dias > 0),
  CHECK (id_insumo IS NOT NULL OR medicamento_texto IS NOT NULL)
);
```

```
-- ---------------------------------------------------------------------
-- 8. Exames
-- ---------------------------------------------------------------------
CREATE TABLE laboratorio (
  id_laboratorio   INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  razao_social     VARCHAR(120) NOT NULL,
  cnpj             CHAR(14)     NOT NULL UNIQUE,
  email_integracao VARCHAR(120),
  ativo            BOOLEAN      NOT NULL DEFAULT TRUE
);

CREATE TABLE tipo_exame (
  id_tipo_exame INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_servico    INT          NOT NULL REFERENCES servico (id_servico),
  nome          VARCHAR(100) NOT NULL UNIQUE,
  material      VARCHAR(40)  NOT NULL,        -- sangue, urina, fezes
  prazo_dias    SMALLINT     CHECK (prazo_dias > 0)
);

CREATE TABLE documento (
  id_documento INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_animal    INT          NOT NULL REFERENCES animal (id_animal),
  tipo         VARCHAR(15)  NOT NULL
               CHECK (tipo IN ('LAUDO','EXAME','TERMO','IMAGEM','OUTRO')),
  nome_arquivo VARCHAR(200) NOT NULL,
  caminho      VARCHAR(400) NOT NULL,         -- chave no repositório de
arquivos
  hash_sha256  CHAR(64)     NOT NULL,         -- garante a integridade do
arquivo
  enviado_por  INT          NOT NULL REFERENCES usuario (id_usuario),
  enviado_em   TIMESTAMPTZ  NOT NULL DEFAULT now()
);

CREATE TABLE solicitacao_exame (
  id_solicitacao   INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_atendimento   INT         NOT NULL REFERENCES atendimento
(id_atendimento),
  id_tipo_exame    INT         NOT NULL REFERENCES tipo_exame
(id_tipo_exame),
  id_laboratorio   INT         REFERENCES laboratorio (id_laboratorio),
  status           VARCHAR(12) NOT NULL DEFAULT 'SOLICITADO'
                   CHECK (status IN
('SOLICITADO','COLETADO','ENVIADO','CONCLUIDO','CANCELADO')),
  coletado_em      TIMESTAMPTZ,
  coletado_por     INT         REFERENCES funcionario (id_funcionario),
```

```
  resultado_em     TIMESTAMPTZ,
  resultado_resumo TEXT,
  id_documento     INT         REFERENCES documento (id_documento),  -- laudo
  CHECK (status <> 'CONCLUIDO' OR resultado_em IS NOT NULL)
);

-- ---------------------------------------------------------------------
-- 9. Programas de doenças crônicas
-- ---------------------------------------------------------------------
CREATE TABLE programa_cronico (
  id_programa            INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_servico             INT         NOT NULL REFERENCES servico
(id_servico),
  nome                   VARCHAR(80) NOT NULL UNIQUE,
  descricao              TEXT,
  intervalo_retorno_dias SMALLINT    NOT NULL CHECK (intervalo_retorno_dias >
0)
);

CREATE TABLE inscricao_programa (
  id_inscricao   INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_animal      INT         NOT NULL REFERENCES animal (id_animal),
  id_programa    INT         NOT NULL REFERENCES programa_cronico
(id_programa),
  id_veterinario INT         NOT NULL REFERENCES veterinario
(id_veterinario),
  inicio         DATE        NOT NULL,
  fim            DATE,
  status         VARCHAR(10) NOT NULL DEFAULT 'ATIVA'
                 CHECK (status IN ('ATIVA','PAUSADA','ENCERRADA')),
  CHECK (fim IS NULL OR fim >= inicio)
);
CREATE UNIQUE INDEX ux_inscricao_ativa
  ON inscricao_programa (id_animal, id_programa) WHERE status = 'ATIVA';

-- ---------------------------------------------------------------------
-- 10. Financeiro
-- ---------------------------------------------------------------------
CREATE TABLE fatura (
  id_fatura      INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_tutor       INT           NOT NULL REFERENCES tutor (id_tutor),
  id_atendimento INT           UNIQUE REFERENCES atendimento
(id_atendimento),
  id_unidade     INT           NOT NULL REFERENCES unidade (id_unidade),
  emitida_em     TIMESTAMPTZ   NOT NULL DEFAULT now(),
  vencimento     DATE          NOT NULL,
```

```
  desconto       NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (desconto >= 0),
  status         VARCHAR(10)   NOT NULL DEFAULT 'ABERTA'
                 CHECK (status IN ('ABERTA','PARCIAL','PAGA','CANCELADA'))
);

CREATE TABLE item_fatura (
  id_item_fatura INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_fatura      INT           NOT NULL REFERENCES fatura (id_fatura),
  id_servico     INT           NOT NULL REFERENCES servico (id_servico),
  quantidade     NUMERIC(8,2)  NOT NULL DEFAULT 1 CHECK (quantidade > 0),
  valor_unitario NUMERIC(10,2) NOT NULL CHECK (valor_unitario >= 0)  -- preço
do dia (RN11)
);

CREATE TABLE pagamento (
  id_pagamento       INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_fatura          INT           NOT NULL REFERENCES fatura (id_fatura),
  forma              VARCHAR(15)   NOT NULL
                     CHECK (forma IN
('PIX','CREDITO','DEBITO','DINHEIRO','BOLETO','CONVENIO')),
  valor              NUMERIC(10,2) NOT NULL CHECK (valor > 0),
  parcelas           SMALLINT      NOT NULL DEFAULT 1 CHECK (parcelas BETWEEN
1 AND 12),
  codigo_autorizacao VARCHAR(40),             -- NSU/ID da transação; nunca o
cartão
  id_convenio        INT           REFERENCES convenio (id_convenio),
  pago_em            TIMESTAMPTZ   NOT NULL DEFAULT now(),
  -- convênio informado se, e somente se, a forma for CONVENIO (RN12)
  CHECK ((forma = 'CONVENIO') = (id_convenio IS NOT NULL))
);

CREATE TABLE nota_fiscal (
  id_nota            INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_fatura          INT           NOT NULL UNIQUE REFERENCES fatura
(id_fatura),
  numero             VARCHAR(15)   NOT NULL,
  serie              VARCHAR(5)    NOT NULL,
  codigo_verificacao VARCHAR(50)   NOT NULL,  -- NFS-e
  emitida_em         TIMESTAMPTZ   NOT NULL,
  valor              NUMERIC(10,2) NOT NULL CHECK (valor >= 0),
  UNIQUE (numero, serie)
);

-- ---------------------------------------------------------------------
-- 11. Auditoria automática (RN15)
-- A aplicação informa o usuário logado com:  SET LOCAL svp.id_usuario =
```

```
'42';
-- Leituras (SELECT) são registradas pela aplicação com acao = 'LEITURA'.
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION fn_auditar() RETURNS trigger AS $$
DECLARE
  linha JSONB := CASE WHEN TG_OP = 'DELETE' THEN to_jsonb(OLD) ELSE
to_jsonb(NEW) END;
BEGIN
  INSERT INTO log_auditoria (id_usuario, acao, tabela, id_registro,
dados_antes, dados_depois)
  VALUES (
    NULLIF(current_setting('svp.id_usuario', true), '')::INT,
    CASE TG_OP WHEN 'INSERT' THEN 'INSERCAO' WHEN 'UPDATE' THEN 'ALTERACAO'
ELSE 'EXCLUSAO' END,
    TG_TABLE_NAME,
    (linha ->> TG_ARGV[0])::BIGINT,
    CASE WHEN TG_OP <> 'INSERT' THEN to_jsonb(OLD) END,
    CASE WHEN TG_OP <> 'DELETE' THEN to_jsonb(NEW) END
  );
  RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_audit_tutor       AFTER INSERT OR UPDATE OR DELETE ON tutor
  FOR EACH ROW EXECUTE FUNCTION fn_auditar('id_tutor');
CREATE TRIGGER tg_audit_animal      AFTER INSERT OR UPDATE OR DELETE ON
animal
  FOR EACH ROW EXECUTE FUNCTION fn_auditar('id_animal');
CREATE TRIGGER tg_audit_alergia     AFTER INSERT OR UPDATE OR DELETE ON
alergia
  FOR EACH ROW EXECUTE FUNCTION fn_auditar('id_alergia');
CREATE TRIGGER tg_audit_triagem     AFTER INSERT OR UPDATE OR DELETE ON
triagem
  FOR EACH ROW EXECUTE FUNCTION fn_auditar('id_triagem');
CREATE TRIGGER tg_audit_atendimento AFTER INSERT OR UPDATE OR DELETE ON
atendimento
  FOR EACH ROW EXECUTE FUNCTION fn_auditar('id_atendimento');
CREATE TRIGGER tg_audit_diagnostico AFTER INSERT OR UPDATE OR DELETE ON
atendimento_diagnostico
  FOR EACH ROW EXECUTE FUNCTION fn_auditar('id_atendimento');
CREATE TRIGGER tg_audit_evolucao    AFTER INSERT OR UPDATE OR DELETE ON
evolucao
  FOR EACH ROW EXECUTE FUNCTION fn_auditar('id_evolucao');
CREATE TRIGGER tg_audit_prescricao  AFTER INSERT OR UPDATE OR DELETE ON
prescricao
  FOR EACH ROW EXECUTE FUNCTION fn_auditar('id_prescricao');
```

```
CREATE TRIGGER tg_audit_exame       AFTER INSERT OR UPDATE OR DELETE ON
solicitacao_exame
  FOR EACH ROW EXECUTE FUNCTION fn_auditar('id_solicitacao');
CREATE TRIGGER tg_audit_documento   AFTER INSERT OR UPDATE OR DELETE ON
documento
  FOR EACH ROW EXECUTE FUNCTION fn_auditar('id_documento');
CREATE TRIGGER tg_audit_pagamento   AFTER INSERT OR UPDATE OR DELETE ON
pagamento
  FOR EACH ROW EXECUTE FUNCTION fn_auditar('id_pagamento');

-- ---------------------------------------------------------------------
-- 12. Visões (valores derivados e indicadores estratégicos)
-- ---------------------------------------------------------------------
CREATE VIEW vw_fatura_total AS
SELECT f.id_fatura,
       COALESCE(SUM(i.quantidade * i.valor_unitario), 0) - f.desconto AS
valor_total
FROM fatura f
LEFT JOIN item_fatura i ON i.id_fatura = f.id_fatura
GROUP BY f.id_fatura, f.desconto;

CREATE VIEW vw_saldo_lote AS
SELECT l.id_lote, l.id_insumo, l.id_unidade, l.validade,
       COALESCE(SUM(CASE WHEN m.tipo IN ('ENTRADA','AJUSTE_ENTRADA')
                         THEN m.quantidade ELSE -m.quantidade END), 0) AS
saldo
FROM lote l
LEFT JOIN movimentacao_estoque m ON m.id_lote = l.id_lote
GROUP BY l.id_lote, l.id_insumo, l.id_unidade, l.validade;

-- objetivo: reduzir o tempo médio de espera
CREATE VIEW vw_tempo_espera AS
SELECT ag.id_unidade,
       date_trunc('month', ag.data_hora) AS mes,
       AVG(atd.inicio_em - ag.chegada_em) AS espera_media
FROM agendamento ag
JOIN atendimento atd ON atd.id_agendamento = ag.id_agendamento
WHERE ag.chegada_em IS NOT NULL
GROUP BY ag.id_unidade, date_trunc('month', ag.data_hora);

-- objetivos: mais animais atendidos por mês e expansão da telemedicina
CREATE VIEW vw_atendimentos_mes AS
SELECT ag.id_unidade,
       date_trunc('month', atd.inicio_em) AS mes,
       ag.modalidade,
       COUNT(*)                    AS atendimentos,
```

```
       COUNT(DISTINCT ag.id_animal) AS animais_atendidos
FROM atendimento atd
JOIN agendamento ag ON ag.id_agendamento = atd.id_agendamento
GROUP BY ag.id_unidade, date_trunc('month', atd.inicio_em), ag.modalidade;

-- ---------------------------------------------------------------------
-- 13. Permissões do banco
-- ---------------------------------------------------------------------
CREATE ROLE svp_app NOLOGIN;   -- papel usado pela aplicação
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO
svp_app;
-- o log só recebe inclusões
REVOKE UPDATE, DELETE ON log_auditoria FROM svp_app;
-- prontuário não se apaga (RN08)
REVOKE DELETE ON triagem, atendimento, atendimento_diagnostico, evolucao,
                 prescricao, item_prescricao, solicitacao_exame, documento
  FROM svp_app;

-- ---------------------------------------------------------------------
-- 14. Dados iniciais
-- ---------------------------------------------------------------------
INSERT INTO perfil (nome, descricao) VALUES
  ('VETERINARIO', 'Prontuário completo dos animais atendidos'),
  ('ENFERMAGEM',  'Triagem, evolução, coleta e baixa de estoque'),
  ('RECEPCAO',    'Cadastro, agenda e recebimento'),
  ('FINANCEIRO',  'Faturamento, pagamentos e notas fiscais'),
  ('DIRETORIA',   'Relatórios agregados'),
  ('AUDITORIA',   'Leitura de registros e do log'),
  ('ADMIN_TI',    'Usuários e perfis, sem acesso a dados de negócio');

INSERT INTO especie (nome) VALUES
  ('Canina'), ('Felina'), ('Equina'), ('Bovina'), ('Ovina'), ('Caprina'),
('Suína'), ('Ave');
```

8. Como o modelo trata os problemas e riscos levantados

Cada problema e risco da etapa 1 tem uma resposta no próprio banco; só backup e disponibilidade dependem da infraestrutura.

Problema ou risco	Solução no modelo
Cadastros duplicados	CPF único em tutor; microchip único em animal; índice único (tutor, espécie, nome) para animais vivos

Problema ou risco	Solução no modelo
Prontuários incompletos ou não padronizados	Campos obrigatórios (anamnese, queixa principal); catálogos de diagnóstico, exame e insumo no lugar de texto livre; status com valores fixos
Dados desatualizados	contato_validado_em e atualizado_em em tutor; a recepção confirma o contato a cada agendamento
Integração com os sistemas de pagamento	pagamento guarda forma, código de autorização e operadora; nota_fiscal guarda o código de verificação da NFS-e; preços vêm de uma única tabela (servico)
Vazamento de dados de saúde	Perfis de acesso por módulo; senhas só em hash; nenhum número de cartão armazenado
Acesso indevido a prontuários	perfil_permissao limita quem lê o prontuário; cada leitura e alteração vai para log_auditoria
Uso para finalidade não autorizada	consentimento registrado por finalidade, com data de aceite e revogação
Perda de dados por falha de backup	Fora do modelo: backup diário com teste de restauração; o hash em documento confirma a integridade dos arquivos restaurados
Indisponibilidade do sistema	Fora do modelo: réplica do banco em outra zona da nuvem no Brasil

Os objetivos estratégicos também viram consultas prontas: `vw_tempo_espera` mede a espera entre a chegada e o início do atendimento, e `vw_atendimentos_mes` conta animais atendidos e teleconsultas por mês e unidade.